



VYNL

Discover. Rate. Connect.

PLANNING & DESIGN DOCUMENT

VYNL MUSIC DISCOVERY, RATING & COMMUNITY PLATFORM

Ivant Wambo
(ST10448302) GR3
Tumelo Teka
(ST10126814) G15
Fezile Jam-Jam
(ST10399080) GR1s



Table of Contents

Introduction.....	4
Overview Of VYNL.....	5
App Requirements.....	8
1.1 Account and Authentication	8
1.2 Single sign-on (SSO), PoE requirement	8
2. Settings	9
3. REST API and Database	9
4. Offline Mode with Sync.....	10
5. Real-Time Push Notifications.....	10
6. Multi-Language Support	11
7. Discovery and Browsing	11
7.1 Dual score system	12
7.2 Two separate rankings	12
7.3 Browse by genre	13
Dedicated genre landing pages rather than a flat tag row: selecting "Genre: Alternative" opens a page with its own Top Rated and Trending sections scoped to that genre, matching IMDb's per-genre browse pages instead of a simple filter chip.....	13
7.4 Advanced search and filters.....	13
Beyond the basic search bar above, an advanced filter panel narrows by genre, release year range, minimum Community Score, and track count, mirroring IMDb's advanced title search.....	13
7.5 New releases and upcoming calendar	13
A chronological release calendar split into "New This Week" and "Upcoming," modelled on Rotten Tomatoes' "In Theaters" and "Coming Soon" split, adapted to release dates instead of theatrical windows.	13
7.6 Editorial curated lists.....	13
Staff-curated collections such as "Best Albums of 2026 So Far" or "Essential First Listens," separate from the user-generated custom lists in section 8. These are maintained through a lightweight "curator" role on the team's own admin tooling, matching Rotten Tomatoes' editorial "Best Of" guides.	13
7.7 Artist pages.....	13
Artist Detail leads with a small "Known For" row of the artist's three or four highest Community Score releases; the same pattern IMDb uses to lead an actor's page with their best-known	

credits before the full filmography. Below that, the discography is listed chronologically, and a "Similar Artists" row (section 7.8) plays the role IMDb's "more like this" row plays on a title page.

.....	13
7.8 Similar-artist recommendations	13
8. Tracking and Library Features	14
9. Social and Community Features	14
10. Personalization and Profile Features	15
11. Gamification Features	15
12. Accessibility and Usability Features	15
13. Tier Lists	16
14. Taste Compatibility	16
15. Daily Challenge and Leaderboard	17
16. Non-Functional Requirements	17
User Interface Design	19
Login	19
Home Feed	21
Album Detail	23
Profile	25
Settings	27
VNYL's API	29
Purpose Of The API	30
Data Sent by The Application	31
User Registration & Login	31
User Profile	31
Album Ratings	31
Song Ratings	32
Reviews	32
Listening Lists	32
Tier Lists	33
Taste Compatibility	33
Data Returned by the API	33
REST API Endpoints	34
External API Integration	35
Backend Development	35

Hosting.....	36
Authentication.....	37
Database.....	37
Database And Storage Technologies	37
1. Firebase Cloud Firestore – NoSQL Database	38
2. Room Database/SQLite (Local Database)	38
3. Azure Blob Storage (Object/Blob Storage)	39
External Music Data.....	39
UML Diagram.....	40
Required Data.....	41
A. User Management Entity (User_Profile)	41
B. Music Metadata & Catalog Entity (Track_Metadata / Album_Metadata).....	42
C. Ratings & Reviews Entity (User_Reviews)	43
D. User Lists & Collections Entity (Custom_Lists).....	44
Project Plan.....	45
Conclusion	46
References	47
Write-Up.....	49

Introduction

The Research Conducted in The Previous Phase of This Project Provided Valuable Insight into The Design, Functionality and User Experience of Three Established Music Applications: Last.fm, Musicboard and Rate Your Music. By Analyzing The Strengths, Weaknesses, and Innovative Features of These Applications, A Clear Understanding Was Developed of The Functionality and Design Principles That Contribute to An Engaging and Effective Music Platform. The Findings from The Research Have Served as The Foundation for The Planning and Design of The Proposed Application, Ensuring That Informed Decisions Are Made Before Development Begins.

This Document Presents the Complete Design Specification for Our Music Discovery, Tracking and Recommendation Application. It Begins with A Brief Overview of The Proposed Application, Including Its Purpose, Branding and The Innovative Features That Distinguish It from Existing Solutions. This Is Followed by A Detailed List of Functional and Non-Functional Requirements That Define the Expected Behavior and Capabilities of The Application.

The Document That Presents the User Interface Design, Including Mock-Ups for Each Screen, Descriptions of Each Screen's Purpose and A Navigation Diagram Illustrating How Users Will Move Throughout the Application. The System Architecture Is Further Explained Through the Design of The REST API, Including How It Will Be Developed, The Hosting Platform That Will Be Used and The Database That Will Store and Manage Application Data. UML Diagrams Are Included to Demonstrate How the API Interacts with The Application's Various Components, While A Detailed Data Specification Identifies the Information That Will Be Captured from Users and Stored Within the Database, Together with The Corresponding Data Types. Finally, The Document Concludes with A Project Plan Presented as A Gantt Chart, Outlining the Milestones and Deadlines That Will Guide the Development Process, Before Ending with A Conclusion and A Complete List of References.

The Purpose of This Planning and Design Document Is to Establish a Comprehensive Blueprint for The Application's Development, Ensuring That Every Component Has Been Carefully Considered Before Implementation Begins. By Translating The Finding of The Research Phase into A Structured System Design, This Document Provides a Clear Roadmap for Building a Functional, User-Friendly and Scalable Music Application.

Overview Of VYNL



Figure 1: VYNL Application Logo

Source: Developed by the author (2026).

VYNL Is an Innovative Album and Song Discovery, Tracking, Review, Rating and Recommendation Application Designed to Provide Music Enthusiasts with A Central Platform for Discovering New Music, Organizing Their Listening Experience, Sharing Opinions and Connecting with A Community of Users Who Share Similar Musical Interests. Inspired By Leading Music Platforms Such as Last.fm, Musicboard and Spotify, VYNL Combines the Strongest Features of These Applications While Introducing Several Original Features That Enhance User Engagement, Personalization and Community Interaction.

Modern Music Streaming Platforms Excel at Providing Users with Access to Millions of Songs; However, They Often Place Less Emphasis on Helping Users Build Meaningful Collections, Express Their Musical Opinions, Discover Music Through Trusted Communities and Track Their Listening Journey in An Engaging Way. VYNL Addresses This Gap by Offering a Comprehensive Platform Where Users Can Discover Albums and Songs, Create Personalized Collections, Rate and Review Music, Monitor Their Listening Activity and Receive Intelligent Recommendations Based on Their Unique Preferences.

The Application Is Designed Around the Philosophy That Every Listener Has a Unique Musical Identity. Rather Than Relying Solely on Popularity-Based Recommendations, VYNL Encourages Users to Actively Participate in The Community by Reviewing Albums, Creating Rankings, Comparing Musical Tastes with

Other Users and Sharing Their Perspectives on Music. By Combining Community-Driven Discovery with Personalized Recommendation System, The Application Aims to Create a Richer and More Engaging Music Discovery Experience.

From A Technical Perspective, VYNL Will Be Developed as A Native Android Application Using Modern Android Development Practices and Industry-Standard Technologies. The Application Will Implement Secure User Authentication, Encrypted Password Storage, RESTful API Communication, Cloud-Based Data Storage and Responsive Material Design User Interfaces to Ensure a Secure, Scalable and User-Friendly Experience. These Technologies Will Enable Users to Seamlessly Synchronize Their Profiles, Reviews, Ratings and Collections Across Devices While Maintaining High Performance and Reliability.

A Distinguishing Characteristics of VYNL Is Its Combinations of Traditional Music Discovery Features with Innovative Community-Driven Functionality. In Addition to Allowing Users to Discover, Review and Rate Albums and Songs, The Application Introduces Several Unique Features That Encourage Creativity, Social Interaction and Long-Term Engagement.

The Innovative Features Planned for VYNL Include:

- **Album Rating System:** Enables Users to Assign Ratings to Albums, Allowing Them to Express Their Opinions While Contributing to Community Ratings That Assist Other Users in Discovering Highly Regarded Music.
- **Album Ranking:** Allows Users to Organize and Rank Their Favorite Albums in Order of Personal Preference, Providing A More Meaningful Representation of Their Musical Tastes Than Individual Ratings Alone.
- **Profile Customization:** Enables Users to Personalize Their Profiles Through Profile Pictures, Biographies, Favorite Artists, Favorites Genres and Other Visual Elements, Creating A Stronger Sense of Identity Within the VYNL Community.
- **Interactive “Taste Compatibility” Index:** Calculates Compatibility Scores Between Users Based on Shared Listening Habits, Favorite Artists, Genres, Album Ratings and Reviews. This Feature Encourages Users to Discover New Friends and Recommendations Through Similar Musical Preferences.
- **Interactive Song & Album Tier Lists:** Allows Users to Create Visually S, A, B, C and D Tier Rankings for Albums or Songs Using an Intuitive Drag-And-Drop Interface. Tier Lists Can Be Customized, Saved to User Profiles and Shared with The Community, Encouraging Discussion, Comparison and Social Engagement. Unlike Traditional Star Ratings, This Feature Enables Users

to Express Nuanced Opinions About an Artist's Discography While Creating Content That Is Highly Shareable Across Social Media Platforms.

- **Daily "Music Wordle" / Album Snippet Challenge:** Introduces A Daily Interactive Challenge Where Users Test Their Music Knowledge by Identifying Albums, Artists or Songs from Clues, Artwork, Metadata or Short Snippets Where Appropriate. Daily Challenges Promote User Retention, Encourage Friendly Competition Through Leaderboards and Provide an Entertaining Way for Users to Engage with The Platform Beyond Standard Music Tracking.

Collectively, These Features Distinguish VYNL From Existing Music Discovery Applications by Combining Music Tracking, Personal Expression, Community Interaction and Gamification Within a Single Ecosystem. The Integration of Innovative Social Features, Personalized Recommendations and Interactive Challenges Transforms the Application from A Simple Music Catalogue into An Engaging Platform Where Users Can Discover Music, Celebrate Their Individual Tastes and Actively Participate in A Vibrant Community of Music Enthusiasts.

The Remainder of This Planning and Design Document Presents the Detailed Requirements, System Architecture, User Interface Designs, Database Structure, API Integration, UML Diagrams, Data Requirements and Project Plan That Will Guide the Successful Development Of VYNL. The Design Decisions Documented Throughout This Report Are Informed by The Research Conducted in The Previous Phase of The Project and Aim to Deliver a Modern, Scalable and User-Centered Application That Meets Both the Functional Requirements of The Project and The Expectations of Contemporary Mobile Application Users.

App Requirements

1.1 Account and Authentication

Register / login with encrypted password, via Firebase Authentication

- **Confirmed with the team:** authentication is delegated entirely to Firebase Authentication, not handled by the app's own REST API. This section is written to match that decision.
- Users create an account with an email and password, or sign in with an existing one, through Firebase Authentication's email/password provider. Firebase Authentication owns credential storage, password hashing, password reset, and email verification; the app's own backend and database never receive or store a raw or hashed password at any point (Firebase, 2026d).
- Minimum password rules are enforced client-side before submission (length of at least 8 characters, at least one number), with inline validation messages, even though Firebase also enforces its own minimum. The app must not crash or fail silently on invalid input; it must tell the user what is wrong.
- **Session handling:** on successful registration or login, Firebase issues an ID token. The app attaches this token to every request to the team's own REST API; the API verifies the token with Firebase before processing the request, rather than issuing or checking a token of its own. The app stores the token securely using Encrypted Shared Preferences or the Android Keystore, not plain Shared Preferences. Token expiry triggers Firebase's own silent refresh.
- Data captured in the app's own database (Fire store) is limited to profile information such as username, display name, bio, and preferences. Email address lives in Firebase Authentication's own record; the app does not duplicate a password_hash field anywhere in its own data model, since no password ever reaches it.

1.2 Single sign-on (SSO), PoE requirement

- Users can select "Continue with Google" alongside the standard email/password form on both the Register and Login screens.
- This is implemented using Firebase Authentication's Google provider (Firebase, 2026a), which handles the OAuth handshake and returns a verified identity token. The app exchanges that for

its own session token from the REST API on first sign-in, creating an account record if one does not already exist, matched by the Google-verified email address.

- SSO accounts still receive a username (auto suggested from the Google display name, editable once), so the rest of the app, including profiles, reviews, and follows, works identically regardless of how the person signed in.

2. Settings

A dedicated Settings screen, reachable from Profile, covers the following:

- **Account:** change username or display name, change password (requires the current password), linked SSO provider status, and delete account, with a confirmation step since this action is destructive and irreversible.
- **Appearance:** a light/dark mode toggle, defaulting to the system setting.
- **Language:** see section 6.
- **Notifications:** per-category toggles (new release alerts; someone rated or reviewed your list; someone followed you); see section 5.
- **Offline and sync** show the current connection state, the size of the pending sync queue, a manual "Sync now" action, and a toggle for whether offline caching downloads album art or text only, as a data-saving option.
- **Connected streaming services:** a placeholder list for future Spotify or Apple Music linking. This is not required for the minimum build, but the settings row and its empty state should exist, so the feature has somewhere to live later.
- **Log out.**

All settings changes are optimistic in the UI, applying immediately, and synced to the API in the background. A failed sync surfaces a small inline retry rather than reverting silently.

3. REST API and Database

- The team builds its own REST API, not just consumes a third-party one. Confirmed with the team: **Firebase Cloud Firestore** is the primary NoSQL database for user-generated and application data (profiles, ratings, reviews, lists, tier lists, compatibility data), backed by the

team's own REST API layer in front of it, while **RoomDB** handles on-device offline storage and the pending-sync queue described in section 4. The API's /auth endpoint is replaced by direct Firebase Authentication calls from the client per section 1.1; the REST API itself only verifies the resulting Firebase ID token on protected requests, it does not issue its own.

- The minimum resources the API must expose are: /users/{id} (profile read/update), /albums, /artists, /tracks (search and detail, which proxy or cache from the Last.fm API rather than hand-authoring a music catalogue) (Last.fm, 2026), /ratings, /reviews, /lists, /follows, /charts (aggregated from ratings), and /notifications.
- Every write endpoint requires a valid Firebase ID token as described in section 1.1; the API rejects unauthenticated or invalid token writes with a 401 response, which the app handles by returning the user to Login rather than crashing.
- Responses are JSON. The app defines typed models matching each resource so that parsing failures are caught rather than left to propagate into a crash.

4. Offline Mode with Sync

- The scope for the minimum build is that users can rate an album, write a review, and add an album to a list while offline. These three actions are the ones most likely to happen in the moment, for example right after listening with no signal, and are the clearest demonstration of the requirement. The rest of the app, such as browsing, charts, and search, can show a "you are offline, some content may be outdated" state instead of full offline browsing.
- Implementation: a local RoomDB (or SQLite) table mirrors the shape of the pending action (rating, review, or list addition) with a synced boolean and a timestamp (Google, 2026a). Actions taken offline write to this table immediately and update the UI optimistically as if they had already succeeded.
- On reconnect, detected using ConnectivityManager or a network callback, a background sync worker built with WorkManager (Google, 2026b) walks the unsynced rows, posts each to the API in order, and marks them synced on success. A conflict, for example if the album was deleted server-side, surfaces as a dismissible notice rather than a silent drop.
- The Settings screen's "offline and sync" section (see section 2) is the single place a user can see this queue and force a retry. Sync status should not be buried only in a toast that disappears.

5. Real-Time Push Notifications

- Implemented using Firebase Cloud Messaging (Firebase, 2026b).
- Notification categories, each with its own settings toggle as described in section 2: a followed artist releases a new album; someone follows the user; someone rates or comments on a list the user created.
- Tapping a notification deep-links into the relevant screen, such as the album detail, the follower's profile, or the list, rather than opening the app to Home. Deep linking is part of the requirement, not optional polish.
- The in-app Notification's screen keeps a scrollable history independent of whether the push arrived while the app was in the foreground, backgrounded, or closed. The API persists notifications server-side; FCM is only the delivery mechanism for the live alert.

6. Multi-Language Support

- The minimum is two South African languages beyond English. isiZulu and Afrikaans are the two chosen here, as they are the most widely spoken official languages after English and have the most complete community translation tooling available.
- Implemented with Android's standard resource qualifiers (`values/`, `values-zu/`, `values-af/` string resources), switchable from Settings, Language, rather than relying only on system locale, since a user may want the app in a language different from their phone's system language (Google, 2026c).
- All user-facing strings, including error messages, empty states, and notification text, live in the string resource files from the start, not hardcoded in layouts, so translation coverage is complete rather than partial.
- Content the users generate themselves, such as reviews and list titles, is never translated. Only the app's own UI chrome is localized.

7. Discovery and Browsing

- Film and TV databases have already solved the discovery problem this app needs to solve for music, so rather than inventing a browsing model from scratch, this section is built directly on IMDb (Wikipedia, 2026) and Rotten Tomatoes (Rotten Tomatoes, 2026) and adapts what carries over. A few of their features are film-specific and are deliberately left out; both the adaptation and the exclusion are explained below.

- **Home feed:** a blended feed combining an editorial hero (a currently trending or featured release) with a tabbed list (Top, Popular, Trending, Favorites) so users have both a curated entry point and an algorithmic one.
- **Global search:** a single search bar covering albums, artists, tracks, and other users, with tabbed result filtering and a "recent searches" shortcut row.

7.1 Dual score system

Every album displays two separate scores rather than one blended number, directly modelled on Rotten Tomatoes' Tomatometer versus Audience Score split (Rotten Tomatoes, 2026).

- **Community Score:** the aggregated star rating from app users, shown as a percentage for a fast visual read; the same underlying data as the 1-to-5-star system in section 8.
- **Critic Score** (phase 2, pending an aggregation source): pulled from linked professional music press coverage once a suitable source or partner API is identified. Where no critic data exists yet, the album shows a Community Score alone rather than a fabricated critic number.
- **Certified badge:** an album earns a "Community Favorite" badge automatically once it crosses a rating-and-volume threshold, for example an average of at least 4 out of 5 stars from at least 100 ratings, the same certified-threshold pattern Rotten Tomatoes uses for "Certified Fresh" (Rotten Tomatoes, 2026). The badge is algorithmic, not editorially assigned, and is removed again if the score drops below the threshold.

7.2 Two separate rankings

Modelled on IMDb keeping its Top 250, a weighted all-time quality ranking, separate from its popularity meter (Wikipedia, 2026).

- **Top Rated:** albums ranked by a weighted rating that accounts for both the average score and the number of ratings, so a handful of five-star ratings cannot outrank an album with thousands of consistently high ratings, the same weighted approach IMDb applies to its Top 250 (Wikipedia, 2026).
- **Trending:** a separate, faster-moving ranking based on the rate of new ratings, reviews, and plays in the last 24 to 48 hours, independent of the all-time score. An album can be Trending without being Top Rated, and the reverse; the two lists are intentionally allowed to disagree.

7.3 Browse by genre

Dedicated genre landing pages rather than a flat tag row: selecting "Genre: Alternative" opens a page with its own Top Rated and Trending sections scoped to that genre, matching IMDb's per-genre browse pages instead of a simple filter chip.

7.4 Advanced search and filters

Beyond the basic search bar above, an advanced filter panel narrows by genre, release year range, minimum Community Score, and track count, mirroring IMDb's advanced title search.

7.5 New releases and upcoming calendar

A chronological release calendar split into "New This Week" and "Upcoming," modelled on Rotten Tomatoes' "In Theaters" and "Coming Soon" split, adapted to release dates instead of theatrical windows.

7.6 Editorial curated lists

Staff-curated collections such as "Best Albums of 2026 So Far" or "Essential First Listens," separate from the user-generated custom lists in section 8. These are maintained through a lightweight "curator" role on the team's own admin tooling, matching Rotten Tomatoes' editorial "Best Of" guides.

7.7 Artist pages

Artist Detail leads with a small "Known For" row of the artist's three or four highest Community Score releases; the same pattern IMDb uses to lead an actor's page with their best-known credits before the full filmography. Below that, the discography is listed chronologically, and a "Similar Artists" row (section 7.8) plays the role IMDb's "more like this" row plays on a title page.

7.8 Similar-artist recommendations

Beyond the basic search bar above, an advanced filter panel narrows by genre, release year range, minimum Community Score, and track count, mirroring IMDb's advanced title search.

8. Tracking and Library Features

- **Ratings and reviews:** a quick 1-to-5-star rating and an optional written review per album, so casual and engaged users are both served, matching the strength identified in the Musicboard research.
- **Listened log:** a lightweight manual "mark as listened" action separate from rating, for users who want to track what they have heard before deciding how they feel about it.
- **Custom themed lists:** user-built collections organised around a theme, mood, or opinion rather than a linear playlist, which other users can browse and follow, the same pattern as IMDb's user-created Lists feature (Wikipedia, 2026), and distinct from the editorially curated lists in section 7.6.
- **Watchlist:** a "want to listen" list distinct from the rated library, for albums a user has discovered but not yet played, directly modelled on IMDb's Watchlist feature (Wikipedia, 2026).
- **Personal listening statistics:** ratings and review history are turned into a visual summary, including top genres, most-rated artists, and a yearly recap card, similar in spirit to the "Wrapped" style summaries that popularised listening analytics.
- **Auto-generated year in review:** at the end of each year, the app compiles the user's top albums and artists automatically from their own rating history, without any manual curation required.

9. Social and Community Features

- **Follow system:** users can follow other users, which drives both the activity feed and new release notifications.
- **Activity feed:** a feed of ratings, reviews, and new lists from people the user follows, separate from the algorithmic home feed.
- **Comments and likes on reviews:** lightweight social interaction on a review, so written criticism becomes a discussion rather than a one-way post.
- **Public/private profile toggle:** users can choose whether their activity and lists are visible to everyone or only to approved followers.
- **Shareable list links:** a custom list can be shared outside the app via a link that opens directly to that list, encouraging discovery from outside the user base.

- **Review helpfulness voting:** other users can mark a review as helpful, the same lightweight quality signal IMDb and Rotten Tomatoes apply to their own audience reviews, which surfaces the most useful reviews above the rest rather than showing them in strict chronological order.

10. Personalization and Profile Features

- **Editable profile:** bio, avatar, and a small set of pinned favorite albums that the user chooses themselves, rather than an automatically generated "top albums" only.
- **Profile stats:** review count, rating count, follower count, and current listening streak, all visible on the profile itself.
- **Badge showcase:** earned gamification badges (see section 11) are displayed on the profile as a visual record of engagement over time.
- **Theme preference:** light and dark mode, chosen independently of the device's system setting if the user prefers.

11. Gamification Features

- **Listening streaks:** a counter for consecutive days with at least one rating or review, visible on the profile
- **Milestone badges:** awarded for events such as the first review, ten albums rated, or a first followed list, shown on the profile's badge showcase.

12. Accessibility and Usability Features

- **Font scaling:** all text respects the system's font scale setting rather than using fixed pixel sizes that ignore accessibility settings.
- **Screen reader support:** every interactive element has a content description, and custom components (such as the star rating control) expose their state to accessibility services, not just their visual appearance.
- **Color contrast:** the design token palette is checked against WCAG AA to contrast minimums for text on its paired background before implementation, not adjusted after the fact.

- **Input validation and error recovery:** every form validates before submission, and every network call has a defined failure state in the UI, so invalid input or a lost connection never crashes into the app.
- **Empty states as guidance:** an empty state (for example, a profile with no activity yet) reads as an invitation to act, with a direct call to action, rather than a blank screen.

13. Tier Lists

- **What this is:** users build S/A/B/C/D tier rankings for either songs or albums drawn from a single artist's discography, using a drag-and-drop interface. This is distinct from the custom themed lists in section 8, which are ordered but not tiered, and from the editorial curated lists in section 7.6, which the user cannot edit.
- **Screen:** a dedicated Tier List builder, opened from an artist's discography page or from the user's own saved tier lists. Five horizontal tier rows, S at the top down to D at the bottom, each acting as a drop target; unplaced tracks or albums sit in a tray below the tiers until dragged into one.
- **Data:** a tier list needs a name, the artist it is scoped to, and an ordered set of item IDs per tier. The shared API design already defines endpoints for this (POST /tierlists, PUT /tierlists/{id}, DELETE /tierlists/{id}) but no corresponding entity in the data specification yet. The app's own model should follow the same shape as the Custom_Lists entity, an owner ID, a title, and an ordered array of item IDs, with an added tier value per item, so the gap in the shared document does not become a gap in what actually gets built.
- **Sharing:** a completed tier list can be shared the same way a custom list can, through the shareable list links described in section 9, since its value comes from being posted or compared rather than kept private.
- **Note for the mockup:** drag-and-drop cannot be demonstrated in a static image, so the mockup shows a tier list mid-build, with one item visibly overlapping the S tier as if partway through being dragged, to communicate the interaction without needing animation.

14. Taste Compatibility

- **What this is:** a percentage score showing how closely two users' music tastes overlap, calculated from shared favorite artists, shared genres, and how similarly the two users have rated albums they have both engaged with.

- **Screen:** shown on another user's profile, never the signed-in user's own, as a compatibility badge near the top of the profile that expands into a short breakdown: shared artists, shared genres, and a small number of albums for both users rated similarly.
- **Data:** the shared API design already defines a `/compatibility/{userId}` endpoint that takes the signed-in user's ID and a target user's ID and returns a calculated score. This section documents what the UI does with that response rather than redefining the calculation itself, which stays the API team's decision to own.
- **Boundary:** a compatibility score is only ever shown between the signed-in user and someone else, never between two other users, and it respects the public/private profile toggle from section 9; a private profile shows no compatibility score to anyone who is not an approved follower.

15. Daily Challenge and Leaderboard

- **What this is:** a once-a-day guessing game, described in the app overview as a "Music Wordle," where users identify an album, artist, or song from a clue, cropped artwork, or a metadata fragment. A shared leaderboard tracks streaks and correct answers across the community.
- **Screen:** A Daily Challenge screen reachable from Home, showing today's puzzle state (unattempt, in progress, or completed), the reveal once solved or given up, and a leaderboard tab ranking users by current streak and all-time correct answers.
- **Data gap to flag:** unlike Tier Lists and Taste Compatibility, this feature has no API endpoint defined anywhere in the shared document yet. Before this screen can be more than a static mockup, the API design needs at minimum an endpoint to fetch today's puzzle and one to submit an answer. Defining that endpoint is outside the scope of this document and should go back to whoever owns the API section.
- **Relationship to section 11's gamification:** this is a distinct, heavier feature than the lightweight optional weekly challenge suggested in section 11. If both ships, the weekly challenge should be folded into this system rather than existing as a second, competing challenge mechanic.

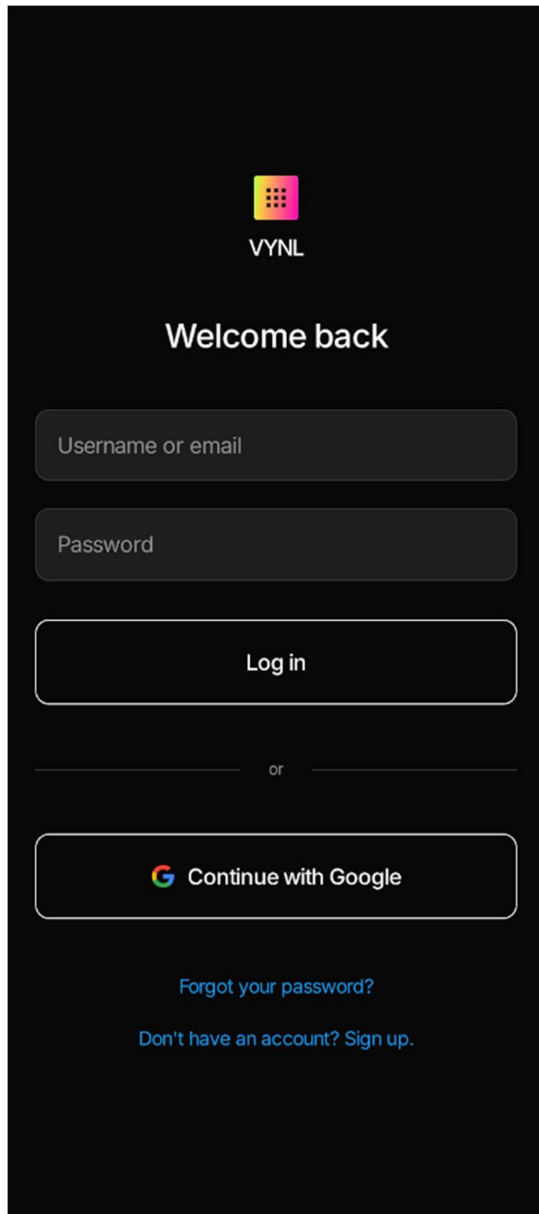
16. Non-Functional Requirements

- The app handles invalid input, such as empty fields, malformed email addresses, or no network connection, without crashing. Every form validates before submission, and every network call has a defined failure state in the UI.

- Consistent use of the design system, meaning tokens, type of scale, spacing, radius, and motion, is maintained across every screen. See the accompanying design system files for the exact values.
- The minimum supported Android version and target SDK are to be confirmed with the lecturer or module guide before Part 2 begins; note the decision here once made so that Part 2's build matches this document.

User Interface Design

Login



A user interface mockup for the VYNL login page. The background is dark. At the top center is the VYNL logo, which consists of a yellow and pink square with a grid of dots, and the text 'VYNL' below it. Below the logo is the text 'Welcome back'. There are two input fields: 'Username or email' and 'Password'. Below these is a 'Log in' button. A horizontal line with the text 'or' in the center separates the login section from the social login section. Below the line is a 'Continue with Google' button. At the bottom, there are two links: 'Forgot your password?' and 'Don't have an account? Sign up.'.

VYNL

Welcome back

Username or email

Password

Log in

or

Continue with Google

[Forgot your password?](#)

[Don't have an account? Sign up.](#)

Figure 2: VYNL Login Page User Interface Mockup (Framer,2026)

Purpose: Entry point for a returning user, and the branch points into Register for a new one. This is the first proof point for the encrypted password and SSO requirements (sections 1.1 and 1.2).

What is on screen: The VYNL mark, a small multi-color grid glyph, sits centered above the wordmark. The only place on this screen color appears at all; everything else is white, gray, or the canvas black. Below that, "Welcome back" introduces two stacked input fields, "Username or email" and "Password," styled as outlined dark fields rather than underlines. A single outlined "Log In" button sits below the fields. An "or" divider separates it from a second, visually identical outlined button, "Continue with Google," carrying the Google "G" mark. Two text links close to the screen: "Forgot your password?" and "Don't have an account? Sign up," both in the system's Signal Blue, which is otherwise unused on this screen. That restraint is deliberate: the blue only ever marks something tappable that leaves the current context.

Why it matters for the requirements: the coexistence of the standard form and the Google button on one screen, rather than two separate flows, is the direct implementation of section 1.2's SSO requirement, an account created through either path ends up with the same username-based profile.

Home Feed

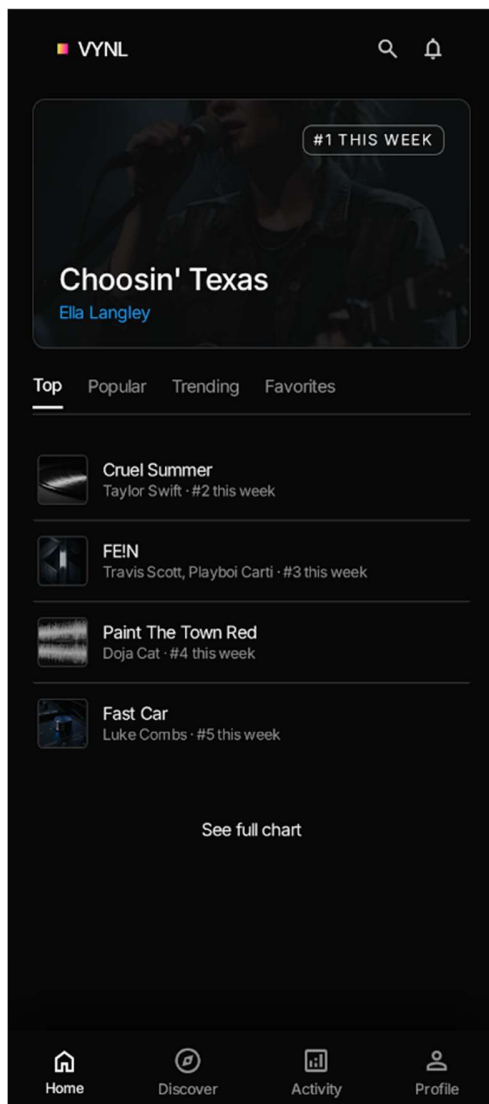


Figure 3: VYNL Home Feed User Interface Mockup (Framer,2026)

Purpose: This acts as the centralized HUB of the Application. It Provides quick access and navigation to any other screen or feature within the application, I.e., profile page, activity page and discover page. It also displays summarized and centralized music info for the user to see on the go.

What is on screen: The VYNL mark, a small multi-color grid glyph, sits centered above the wordmark. On the top-right corner is the search icon as well as the bell icon which is for notifications. Right in the middle on top of the screen is the current 1# song of the week, which in this case is Choosin'Texas by Ella Langley. Under the trending song is for different categories which each display songs that best exemplify the category, e.g., the "Top" category shows the top-rated songs on the billboard. Under the top-rated songs there is a button called "See full chart" which allows the user to see the complete list of top songs upon pressing the button, under that there is a navigation bar which will direct the user to the screen that they select which can either be Home, Discover, Activity and Profile.

Why it matters for the requirements: The home screen is important to VYNL because it serves as the central entry point to the application's core functionality and supports the requirement for an intuitive and user-friendly navigation experience. It provides users with convenient access to music discovery, personalized recommendation, recently viewed or trending albums and songs and other key features of the application. By presenting relevant content in a clear and organized manner it ensures that the application's main requirements are easily accessible from a central location.

Album Detail

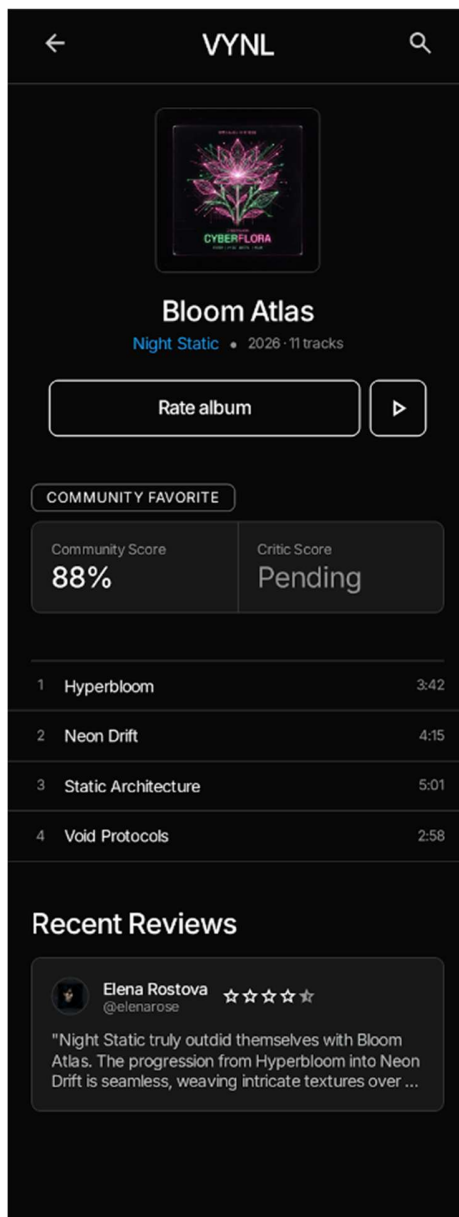


Figure 4: VYNL Album Detail Page User Interface Mockup (Framer,2026)

Purpose: The most feature-dense screen in the app, where a user rates, reviews, and evaluates a specific release. Demonstrates the dual-score system from section 7.1 and the rating/review features from section 8.

What is on screen: A back arrow and the VYNL wordmark sit in a minimal top bar, followed by the album artwork, title ("Bloom Atlas"), and artist name (again a blue link, this time to "Night Static"), with release year and track count underneath. Two buttons follow: an outlined "Rate album" action and a smaller icon-only play button beside it. A "COMMUNITY FAVORITE" tag chip appears above the score block; this is the algorithmic badge described in section 7.1, shown only because this album has crossed the ratings threshold. The score block itself is split into two columns by a hairline divider: "Community Score, 88%" on the left, and "Critic Score, pending" on the right, visibly muted since no critic aggregation source exists yet, exactly the phase-2 treatment specified in the requirements rather than a fabricated number. A numbered track list follows, each row showing a track title and its duration. The screen closes with a "Recent Reviews" section showing one review card, an avatar, reviewer name, a star rating, and the review text itself, demonstrating that both the quick star rating and the written review coexist on the same screen rather than requiring separate flows.

Why it matters for the requirements: this single screen demonstrates section 7.1's dual scoring, the Community Favorite badge logic, and section 8's combined star-rating and written-review pattern all at once, making it the natural screen to lead with when presenting the app's core loop.

Profile

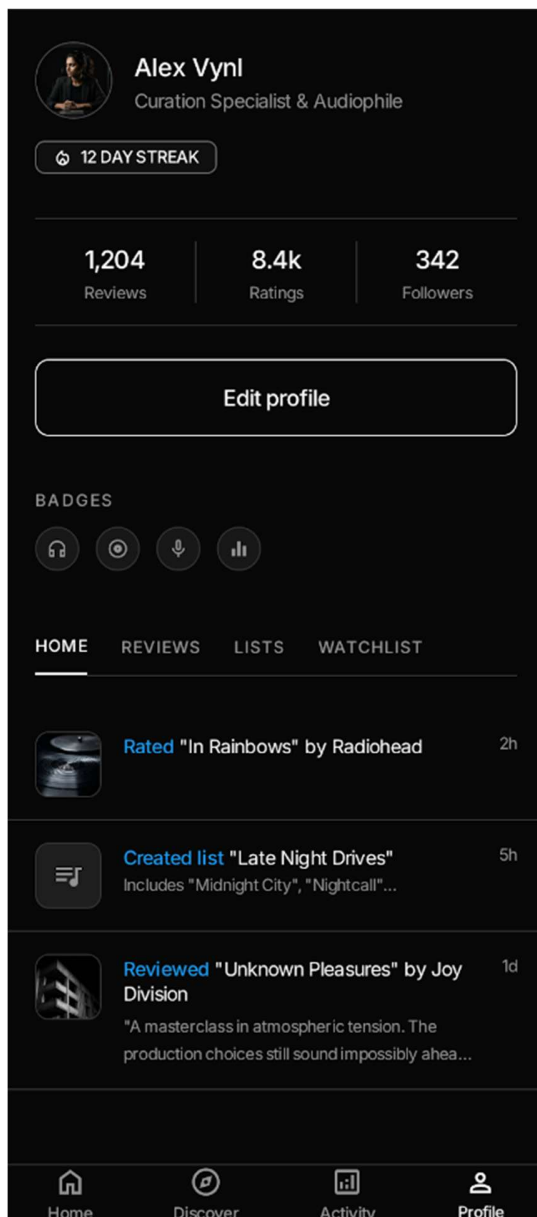


Figure 5: VYNL Profile Page User Interface Mockup (Framer,2026)

Purpose: The user's own identity hub, and the entry points into Settings, Lists, and Watchlist. Demonstrates personalization (section 10), gamification (section 11), and the social activity feed (section 9).

What is on screen: An avatar and display name sits at the top, with a short role-style bio line beneath it. A "12 DAY STREAK" tag chip follows; the gamification streak counter from section 11, styled identically to every other tag chip in the system rather than being given special color treatment. A three-column stats row shows review count, rating count, and follower count side by side, divided by hairline rules rather than separate cards. The outlined "Edit profile" button sits below that. A "BADGES" label introduces a row of circular badge icons; the milestone rewards section 11's gamification requirement, each with a simple outlined glyph rather than an illustrated icon, keeping them consistent with the rest of the system restraint. A four-item tab row, Home, Reviews, Lists, Watchlist, exposes the library features from section 8 directly from the profile. Below the tabs, an activity feed lists the user's own recent actions, a rating, a created list, and a written review, each shown with a relative timestamp. The same feed pattern used for the social Activity Feed in section 9 but scoped to one person here.

Why it matters for the requirements: the streak chip and badge row are the concrete UI for section 11's gamification requirement, kept deliberately understated rather than turned into a rewards dashboard, in keeping with the requirement's own note that gamification should encourage engagement without becoming manipulative.

Settings

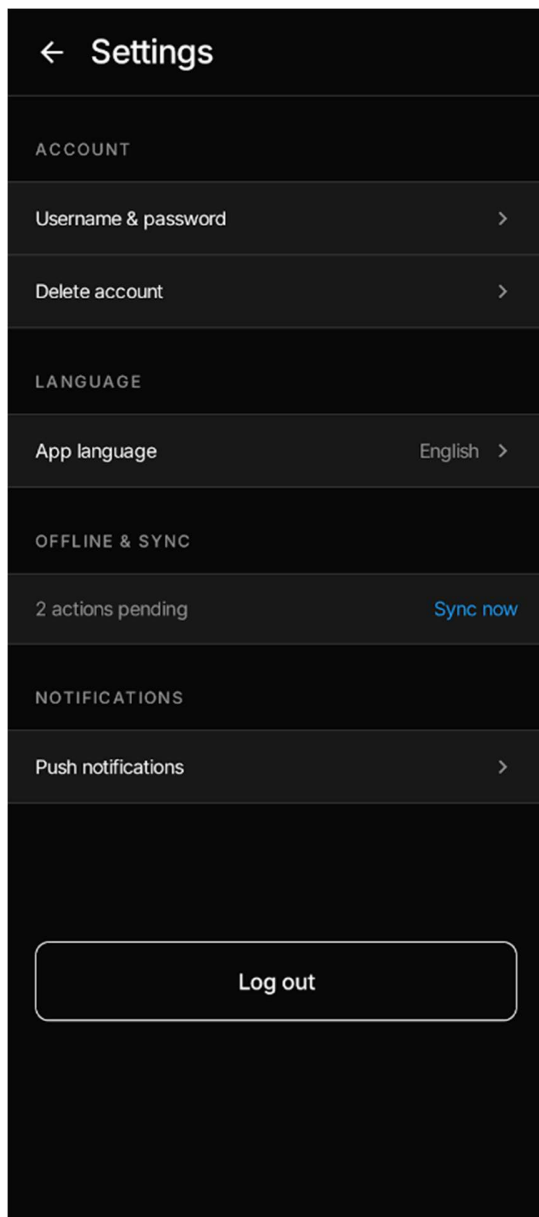


Figure 6: VYNL Settings Page User Interface Mockup (Framer,2026)

Purpose: The single screen that must visibly prove every PoE-only requirement at once: SSO account management, offline sync visibility, push notifications, and multi-language support. Demonstrates section 2 in full.

What is on screen: A back arrow and "Settings" title sit at the top, followed by four labelled sections, each introduced by a small uppercase section label rather than a divider line. "Account" holds "Username & password" and "Delete account," both styled as plain navigational rows rather than one being flagged as destructive, a deliberate consequence of the token system having no error in color, noted separately to the team. "Language" shows "App language" with its current value, "English," and a chevron indicating it opens a picker, the entry point for the isiZulu and Afrikaans requirement in section 6. "Offline & Sync" is the one row that breaks the plain-row pattern on purpose: "2 actions pending" sits on the left in the default text color, and "Sync now" sits on the right in Signal Blue as a tappable action, this is the visible queue and manual-retry control required in section 4. "Notifications" holds a single "Push notifications" row leading to the category toggles described in section 5. The screen closes with an outlined "Log out" button, styled identically to every button elsewhere in the app.

Why it matters for the requirements: this is the only screen where four separate PoE-only requirements, SSO-linked account management, offline sync, notifications, and multi-language, are all visible on one screen at once, which is worth stating explicitly in the document rather than leaving it for the marker to notice.

VYNL Application Screen Navigation Diagram

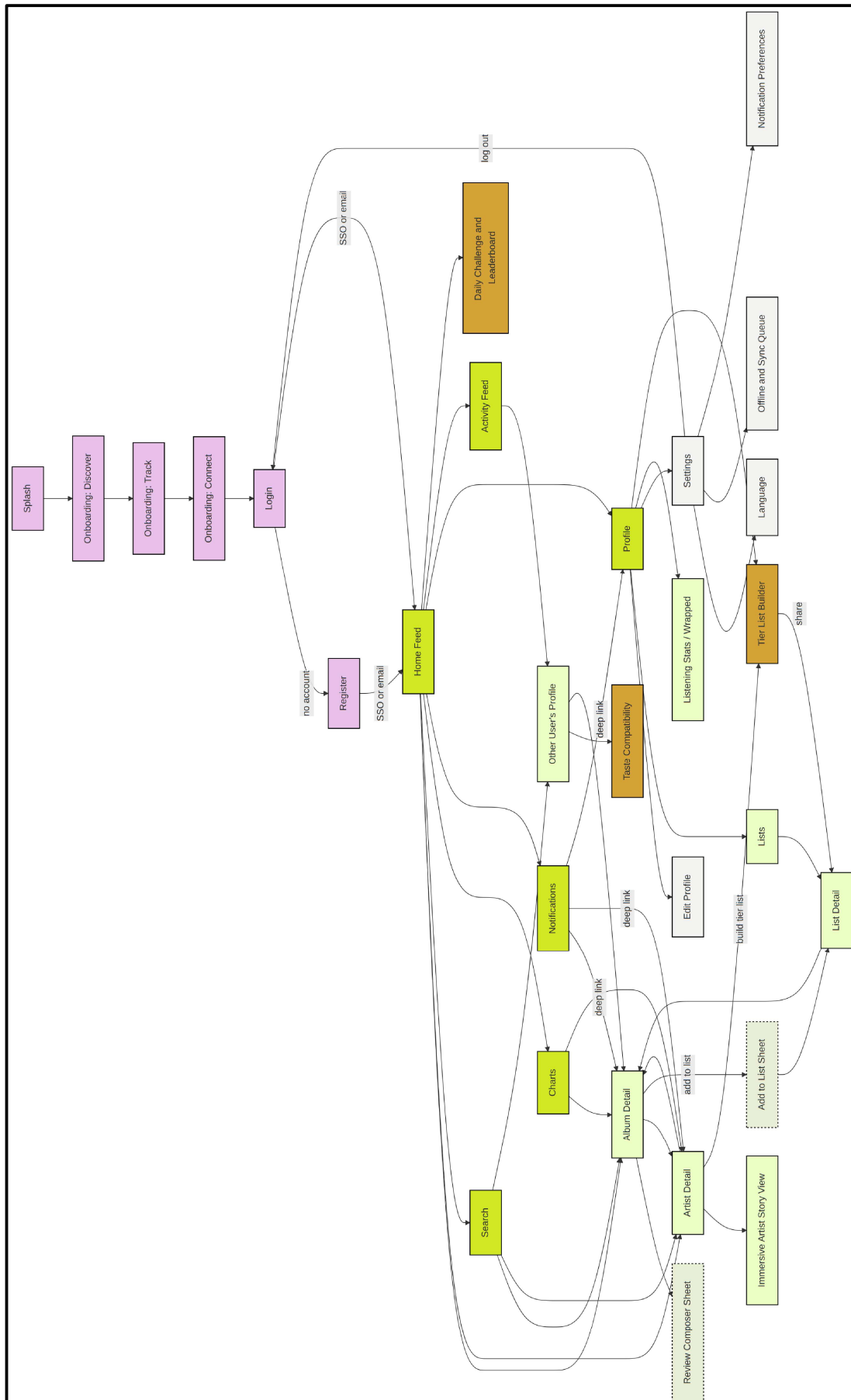


Figure 7: VYNL Application Screen Navigation Diagram [Author's own work created using Mermaid (2026)].

N.B., We Apologize for The Inconvenience Caused by The Diagram's Orientation, This Was The Only Way To Showcase It Completely Without Cropping Any Parts.

VNYL's API

VYNL Will Utilize A RESTful Application Programming Interface (REST API) To Facilitate Secure Communication Between the Android Application, The Backend Server, and The Database. The API Will Act as The Intermediary Responsible for Processing Client Requests, Validating User Input, Retrieving Music Information, Storing User-Generated Content and Returning Responses in JSON Format.

The API Will Be Designed to Separate the Application's Presentation Layer from The Business Logic and Database, Creating A Modular, Scalable and Maintainable System Architecture. This Approach Allows Future Versions of VYNL To Support Additional Platforms, Such as A Web Application Or iOS Application, Without Requiring Changes to The Core Backend Services.

Authentication Will Be Handled Separately Using Firebase Authentication, Which Satisfies the Project Requirement for Secure User Registration and Login. Once Authenticated, Users Will Receive a Secure Firebase ID Token That the Backend API Will Verify Before Processing Authorized Requests.

Purpose Of The API

The VYNL API Will Perform the Following Primary Functions:

- Authenticate And Authorize Users Through Firebase Authentication.
- Retrieve Album, Artist and Song Information from An External Music Database API.
- Store And Manage User-Generated Content.
- Process Album and Song Ratings.
- Store And Retrieve User Reviews.
- Manage Personalized Listening Lists.
- Generate Personalized Music Recommendations.
- Calculate Taste Compatibility Between Users.
- Store And Retrieve Album and Song Tier Lists.

- Manage User Profile Information.
- Retrieve Application Statistics and Analytics.
- Synchronize User Data Across Multiple Devices.

Data Sent by The Application

The Android Application Will Send Data to The API Whenever a User Performs an Action Within the Application.

Examples Include:

User Registration & Login

Authentication Is Managed by Firebase Authentication, Therefore the Application Will Send:

- Email Address
- Password
- Firebase Authentication Token

The API Will Validate the Firebase Token Before Allowing Access to Protected Resources.

User Profile

The Application Will Send:

- Username
- Profile Picture
- Biography
- Favorite Genres
- Favorite Artists

Album Ratings

Users Will Submit:

- Album ID
- Rating (1-5 Stars)
- User ID

Song Ratings

Users Will Submit:

- Song ID
- Rating
- User ID

Reviews

Users May Submit:

- Album ID
- Song ID
- Review Title
- Review Text
- Rating
- Date Created

Listening Lists

Users Can Create Personalized Lists

The Application Will Send:

- List Name
- Description
- Album IDs
- Song IDs
- Visibility (Public/Private)

Tier Lists

For Album Tier Lists:

- Tier List Name
- Artist ID
- Album IDs
- Tier Assignment (S, A, B, C or D)

For Song Tier Lists:

- Tier List Name
- Artist ID
- Song IDs
- Tier Assignment

Taste Compatibility

The API Will Receive:

- Current User ID
- Target Used ID

The Backend Will Calculate Compatibility Based On:

- Shared Favorite Artists
- Shared Genres
- Album Ratings
- Song Ratings
- Reviews
- Listening History

Data Returned by the API

The API Will Return Data in JSON Format.

Example:

```
{
  "username": "MusicFan01",
  "followers": 210,
  "following": 145,
  "reviews": 83,
  "albumRatings": 197,
  "tierLists": 12
}
```

REST API Endpoints

The API Will Expose Several RESTful Endpoints.

HTTP Method	Endpoint	Purpose
POST	/users/profile	Create Or Update User Profile
GET	/users/profile/{id}	Retrieve User Profile
GET	/artists	Retrieve Artists
GET	/albums	Retrieve Albums
GET	/songs	Retrieve Songs
GET	/albums/	Retrieve Album Details
GET	/songs/{id}	Retrieve Song Details
POST	/ratings/album	Submit Album Rating
POST	/ratings/Song	Submit Song Rating
POST	/reviews	Submit Review
GET	/reviews/{albumId}	Retrieve Reviews
POST	/lists	Create Listening List
GET	/lists/{userId}	Retrieve Listening Lists
POST	/tierlists	Create Tier List
PUT	/tierlists/{id}	Update Tier List
DELETE	/tierlists/{id}	Delete Tier List
GET	/recommendations	Retrieve Recommendations
GET	/compatibility/{userId}	Calculate Taste Compatability

Figure 8: Table Displaying RESTful API Endpoints.

Source: Developed by the author (2026).

External API Integration

Rather Than Manually Storing Millions of Songs, Albums and Artists Within the Application's Database, VYNL Will Integrate with The Last.fm API To Retrieve Music Metadata and Enhance the Music Discovery Experience. The Last.fm API Provides Developers with Access to A Comprehensive Catalogue of Music-Related Information, Making It an Ideal External Data Source for The Application.

The Last.fm API Will Be Used to Retrieve:

- Artist Information
- Album Information
- Song Information
- Album Artwork
- Artist Biographies
- Music Genres and Tags
- Top Albums
- Top Tracks
- Similar Artists
- Similar Albums
- Popular Music Charts
- Music Search Results

By Integrating The Last.fm API, VYNL Will Always Display Current and Relevant Music Information Without Requiring the Application to Maintain Its Own Extensive Catalogue of Artists, Albums and Songs. This Significantly Reduces Storage Requirements, Improves Scalability and Ensures That Newly Released Music Can Be Accessed Without Manual Database Updates.

The Android Application Will Communicate with The Custom ASP.NET Core REST API Whenever Music Information Is Requested. Rather Than Allowing the Mobile Application to Communicate Directly with The Last.fm API, The Backend API Will Act as An Intermediary. This Approach Provides Several Advantages, Including Improved Security, Centralized Error Handling, Response Caching, Request Validation and Reduced Exposure of API Credentials.

Backend Development

The REST API Will Be Developed Using ASP.NET Core Web API (C#).

ASP.NET Core Was Selected Because It Offers:

- High Performance
- Cross-Platform Compatibility
- Excellent REST API Support
- Strong Security Features
- Dependency Injection
- Entity Framework Core Integration
- JWT/Firebase Authentication Support
- Excellent Scalability
- Industry Adoption

The API Will Follow a Layered Architecture Consisting Of:

- Controllers
- Services
- Repositories
- Data Access Layer
- Database

This Architecture Improves Maintainability, Testing and Scalability.

Hosting

The REST API Will Be Hosted Using Microsoft Azure App Service.

Azure App Service Was Selected Because It:

- Integrates Seamlessly with ASP.NET Core
- Provides Automatic Scaling
- Offers High Availability
- Supports HTTPS By Default
- Includes Deployment Through GitHub Actions

- Provides Monitoring and Logging
- Supports Continuous Deployment

Hosting The API On Azure Allows Future Expansion of The Application While Ensuring Secure and Reliable Access for Users.

Authentication

User Registration and Login Will Be Implemented Using Firebase Authentication, As Required by The Project Specifications.

Firebase Authentication Will Provide:

- Secure Email and Password Authentication
- Password Encryption and Hashing
- User Identity Management
- Authentication Token Generation
- Session Management
- Password Reset Functionality
- Email Verification

After Successful Authentication, Firebase Will Issue an ID Token That Will Accompany Each Protected API Request. The Backend API Will Verify This Token Before Allowing Users to Access Protected Resources, Ensuring That Only Authenticated Users Can Perform Action Such as Creating Reviews, Rating Albums or Managing Tier Lists.

Database

VYNL Will Make Use of a Combination of Three Storage Technologies, Because Each Satisfies a Different Requirement:

Database And Storage Technologies

1. Firebase Cloud Firestore – NoSQL Database

Firestore Will Be the Primary Cloud-Based NoSQL Database For VYNL. It Will Store Application and User-Generated Data Such As:

- User Profiles
- Album And Song Ratings
- Reviews
- Listening Lists
- Album Rankings
- Song And Album Tier Lists
- User Preferences
- Taste Compatibility Data
- Daily Challenge Information

Firestore Is Appropriate Because VYNL Contains User-Generated and Semi-Structured Data That Can Be Represented Using Documents and Collections.

2. Room Database/SQLite (Local Database)

The Android Application Will Use Room Database, Which Is Built on SQLite, To Provide the Required Offline Functionality. It Will Store Locally Cached Information and Data That Needs to Remain Available When the Device Has No Internet Connection.

Examples include:

- Recently Viewed Albums and Songs
- Cached Music Metadata
- Cached Recommendations
- User Preferences
- Pending Ratings or Reviews Awaiting Synchronization

When Connectivity Is Restored, Appropriate Pending Changes Can Be Synchronized With The Cloud Backend.

3. Azure Blob Storage (Object/Blob Storage)

Azure Blob Storage Will Be Used for Storing Large Binary Files Rather Than Structured Database Records. For VYNL, This Can Include:

- Profile Pictures
- User-Uploaded Images
- Generated Song & Album Tier List Images
- Other Application Media

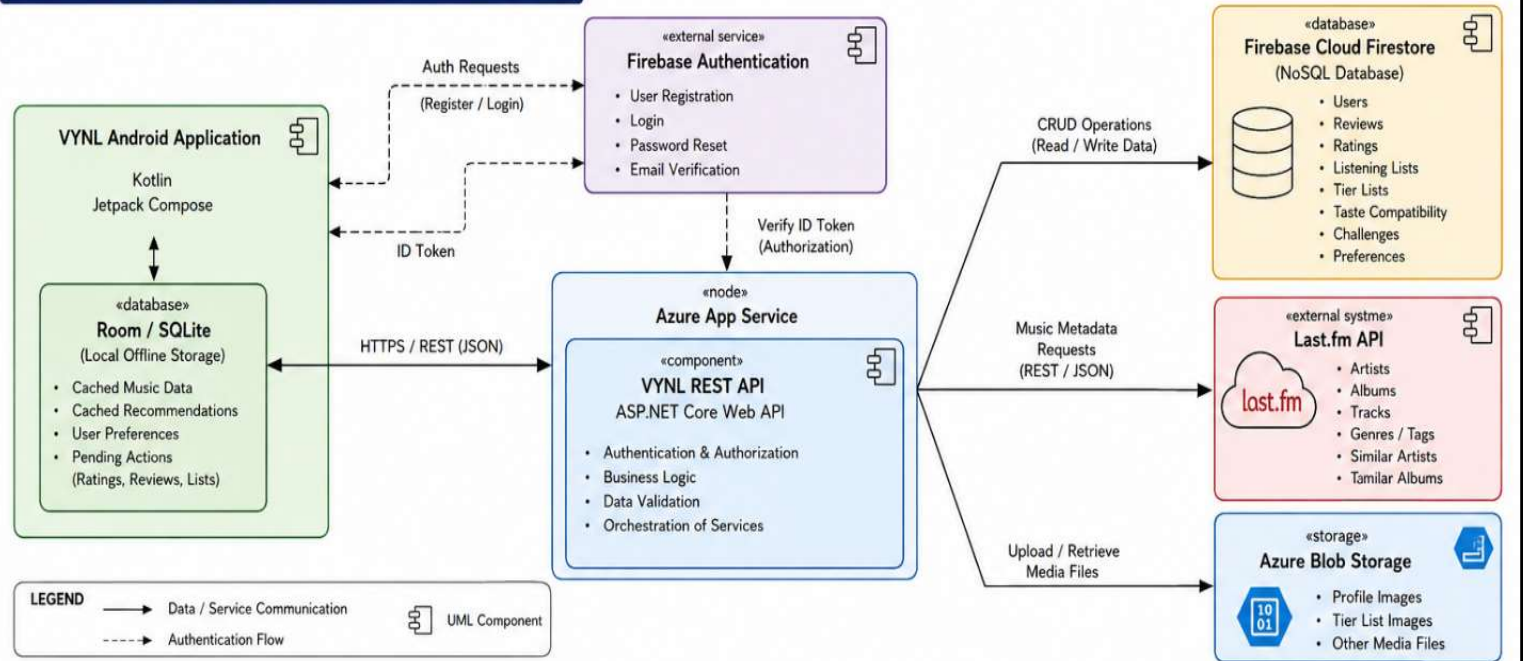
The Actual File Can Be Stored in Blob Storage While Its URL/Reference Can Be Stored Alongside the Relevant Document in Firestore.

External Music Data

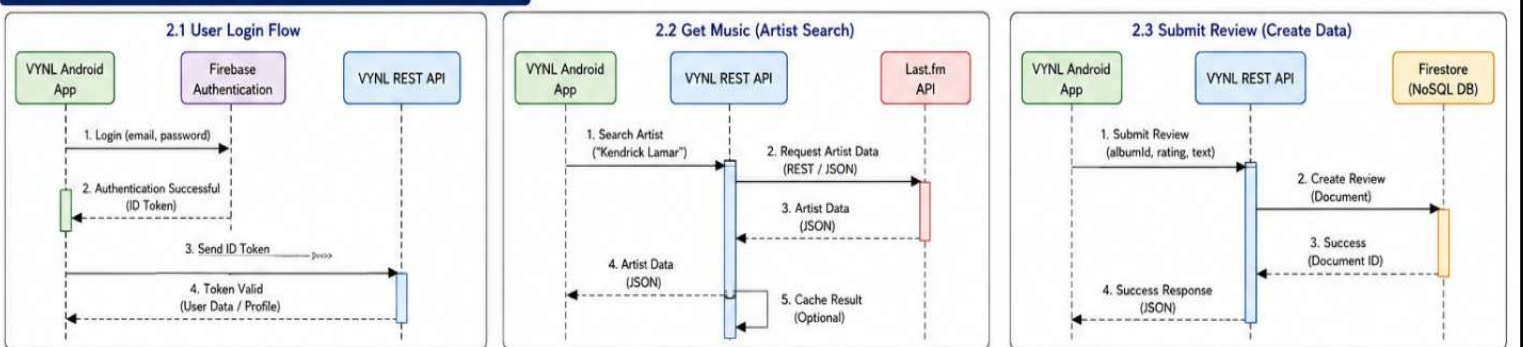
Last.fm API Will Provide the External Music Metadata Rather Than Being Used as VYNL's Database. VYNL Can Retrieve Information Such as Artists, Albums, Tracks, Tags and Related Music from Last.fm, While User-Generated Information Remains Under VYNL's Control.

UML Diagram

1. UML COMPONENT DIAGRAM – VYNL SYSTEM ARCHITECTURE



2. UML SEQUENCE DIAGRAMS – EXAMPLE INTERACTIONS



NOTES

- The Android App communicates with the VYNL REST API over HTTPS using JSON.
- Firebase Authentication issues an ID Token which is verified by the API for secure access.

- Last.fm API provides music metadata (artists, albums, songs, tags, etc.).
- Firestore stores structured NoSQL data for the application.

- Azure Blob Storage stores binary files (images, media).
- Room / SQLite provides offline capability on the device.

Figure 8: VYNL API Component Diagram and System Integration

Source: Developed by the author (2026).

Required Data

To support user accounts, music tracking, social reviews, streaming integration, and app customizability, the system architecture requires a relational/document-oriented database structure (e.g., Firebase Firestore or PostgreSQL/Room). Below is the detailed breakdown of the data to capture and store, along with their respective data types.

A. User Management Entity (User_Profile)

Stores personal information, credentials, and app preferences required for account creation, login, and personalized profiles.

Field Name	Data Type	Description / Purpose
user_id	String (UUID / Primary Key)	Unique identifier generated upon account registration.
username	String (Unique)	Public display name for community interactions.
email	String	Registered email address used for authentication and communications.
password_hash	String	Securely hashed password (managed via Firebase Auth).
profile_picture_url	String (URL)	Link to image stored in Cloud Storage.

bio	String (Nullable)	Short user biography or personal status.
favorite_genres	List<String>	Array of preferred musical genres for initial onboarding.
spotify_connected	Boolean	Flag indicating if the user linked their Spotify Web API account.
created_at	Timestamp	System timestamp when the account was registered.

B. Music Metadata & Catalog Entity (Track_Metadata / Album_Metadata)

Cached metadata retrieved from the Spotify API to optimize app performance and eliminate redundant API calls.

Field Name	Data Type	Description / Purpose
spotify_item_id	String (Primary Key)	Spotify URI / unique ID for the track or album.
title	String	Name of the track or album.
artist_name	String	Name of the primary performing artist.
cover_art_url	String (URL)	Direct CDN link to high-resolution album artwork.

release_date	String / Date	Official release date of the music asset.
track_count	Integer	Total number of songs contained within an album.
preview_url	String (URL)	30-second audio stream URL for in-app listening.

C. Ratings & Reviews Entity (User_Reviews)

Captures community contributions, including star ratings, written critiques, and review metadata.

Field Name	Data Type	Description / Purpose
review_id	String (UUID / Primary Key)	Unique identifier for each published review.
user_id	String (Foreign Key)	Refers to the author's user_id.
spotify_item_id	String (Foreign Key)	Refers to the targeted album or track.
rating_value	Float (e.g., 0.0 to 5.0)	Granular star rating given by the user.
review_text	String (Text / Long)	Detailed written critique or thoughts on the release.

like_count	Integer	Total community likes accumulated on the review.
created_timestamp	Timestamp	Date and time the review was posted.

D. User Lists & Collections Entity (Custom_Lists)

Stores custom-curated user lists, theme rankings, and tier lists.

Field Name	Data Type	Description / Purpose
list_id	String (UUID / Primary Key)	Unique identifier for a curated list.
user_id	String (Foreign Key)	Owner of the custom list.
list_title	String	Descriptive name (e.g., <i>"Top 10 Hip Hop Albums of 2025"</i>).
list_description	String	Contextual text explaining the theme of the list.
item_ids_order	List<String>	Ordered array of spotify_item_ids maintaining ranking position.
is_public	Boolean	Privacy toggle determining whether followers can view the list.

Project Plan

VYNL Project Plan

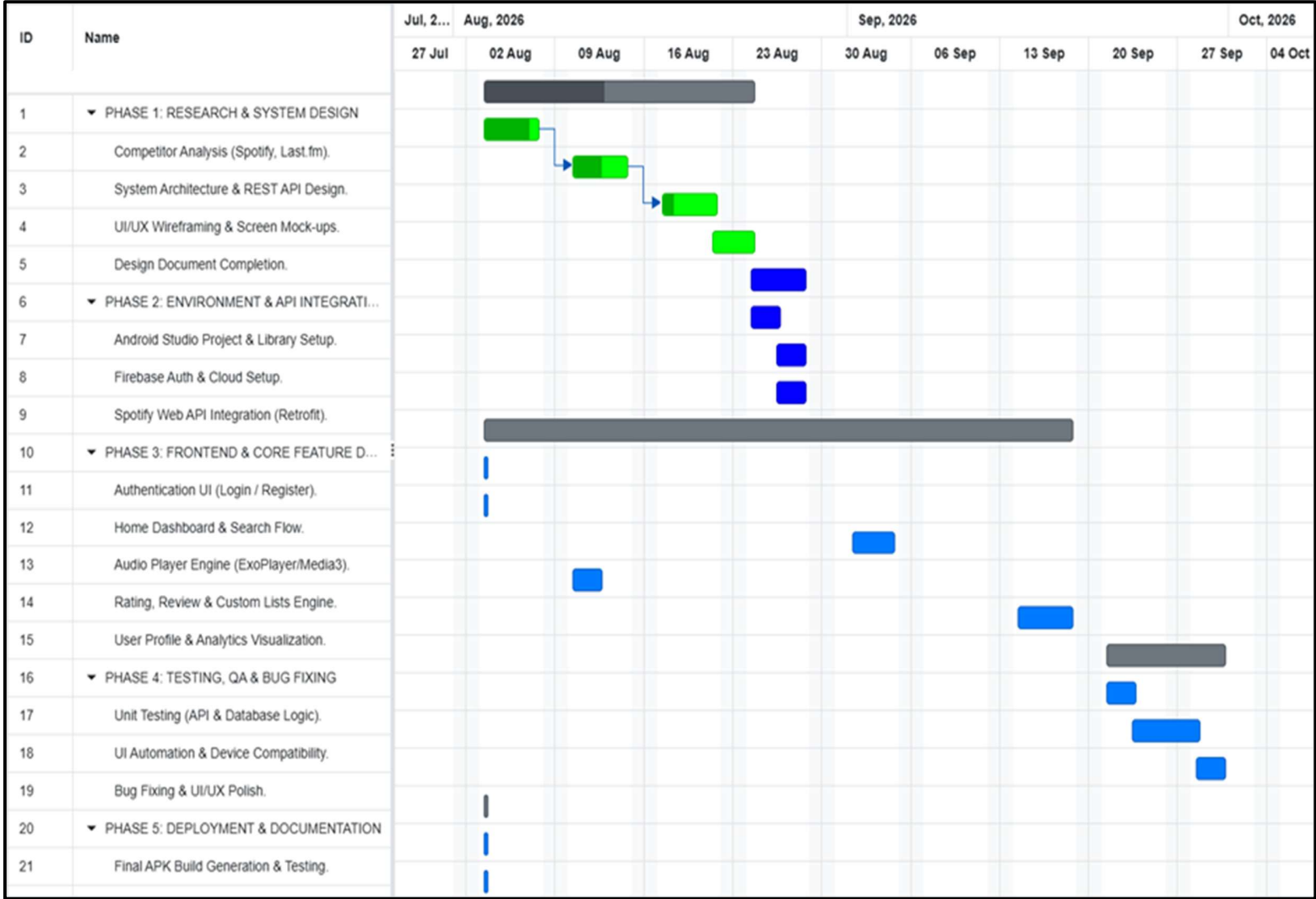


Figure 9: VYNL Project Plan and Gantt Chart

Source: Developed by the author (2026).

Conclusion

The Planning and Design Process Has Transformed the Findings from The Research Phase into A Structured and Practical Blueprint for The Development of The Proposed Music Discovery, Tracking and Recommendation Application. The Analysis of Last.fm, Musicboard and Rate Your Music Provided Valuable Insight into Successful Design Practices, User Expectation, and Innovative Functionality, All of Which Have Influenced the Design Decisions Presented Throughout This Document.

This Document Has Defined the Application's Objectives, Functional and Technical Requirements, User Interface, Navigation Structure, API Architecture, Database Design, UML Diagrams, Data Specifications and Project Timeline. Together, These Elements Establish a Clear Development Plan That Reduces Uncertainty During Implementation While Ensuring That the Application Satisfies Both the Project Requirements and The Needs of Its Intended Users.

With A Detailed Design Now in Place, the Project Is Well Prepared to Move into The Development Phase. The Documentation Created in This Stage Will Serve as A Reference Throughout Implementation, Helping to Ensure Consistency, Maintainability and Alignment with The Original Project Objectives. By Building Upon The Research Conducted in The Previous Document and Following the Plans Outlined Here, The Project Is Positioned to Deliver a Well-Designed, Scalable and User-Centered Music Application.

References

- Android Developers (2026a) *Save data in a local database using Room*. Available at: [Android Developers – Room Database](#) (Accessed: 9 August 2026).
- Android Developers (2026b) *Persist data with Room*. Available at: [Android Developers – Room Database](#) (Accessed: 9 August 2026).
- Firebase (2026a) *Get Started with Firebase Authentication on Android*. Available: [Firebase Authentication for Android](#)
- Firebase (2026b) *Cloud Firestore data model*. Available at: [Firebase Cloud Firestore Data Model](#)
- Firebase (2026c) *Cloud Firestore API*. Available at: [Firebase Cloud Firestore API](#)
- Google (2026c) *Support different languages and cultures*. Available at: <https://developer.android.com/guide/topics/resources/multilingual-support> (Accessed: 5 August 2026).
- Google Play (2026) *Prepare your app for review*. Available at: <https://support.google.com/googleplay/android-developer/answer/9859455> (Accessed: 5 August 2026).
- Last.Fm (2026a) *Last.fm Music Discovery API: Introduction*. Available at: [Last.fm API Documentation](#)
- Last.fm (2026b) *Last.fm API Documentation*. Available at: [Last.fm API Documentation and Methods](#)
- Microsoft (2026c) *ASP.NET Core web API documentation with Swagger/OpenAPI*. Microsoft Learn. Available at: Microsoft Learn - ASP.NET Core Web API (Accessed: 9 August 2026).
- Microsoft (2026b) *Azure App Service documentation*. Microsoft Learn. Available at: [Microsoft Learn – Azure App Service](#) (Accessed: 9 August 2026).
- Microsoft (2026b) *Azure App Service documentation*. Microsoft Learn. Available at: [Microsoft Learn – Azure App Service](#) (Accessed: 9 August 2026).
- Microsoft (2026c) *Introduction to Azure Storage*. Microsoft Learn: Available at: [Microsoft Learn – Azure Storage](#) (Accessed: 9 August 2026).
- Microsoft (2026d) *Blob Storage REST API*. Microsoft Learn. Available at: [Microsoft Learn – Azure Storage](#) (Accessed: 9 August 2026).
- Object Management Group (2017) *OMG Unified Modeling Language (ONG UNL), Version 2.5.1*. Available at: [OMG UML 2.5.1 Specification](#) (Accessed: 9 August 2026).
- <https://ganttchartmaker.com/gantt/#/online>.
- Rotten Tomatoes (2026) *About Rotten Tomatoes*. Available at: <https://www.rottentomatoes.com/about> (Accessed: 5 August 2026).
- W3C Web Accessibility Initiative (2026) *Web Content Accessibility Guidelines (WCAG) 2.1*. Available at: <https://www.w3.org/TR/WCAG21/> (Accessed: 5 August 2026).
- Wikipedia (2026) *IMDb*. Available at: <https://en.wikipedia.org/wiki/IMDb> (Accessed: 5 August 2026).
- Mermaid (2026) *Mermaid Live Editor*. Available at: [Mermaid Live Editor](#) (Accessed: 5 August 2026).

- Framer (2026) *Framer: Design and publish websites*. Available at: [Framer](#) (Accessed: 5 August 2026).

Write-Up

No AI Was Used in This Document.